Who Sees What — False Positives and Accepted Findings

Package	Who Sees What (twinforce)
Publisher	Twinforce Solutions Limited
Contact	info@twinstack.net
Package Id	0HoOW00000000ZV0AY
Version submitted	0.3.0.1 (04tOW00000Gs9l8YAB)
Scanner	Salesforce Code Analyzer plugin 5.15.0 (PMD 7.26.0, RetireJS, Graph Engine)
Date	17 August 2026

Summary

The AppExchange and Security rulesets report no violations:

```
sf code-analyzer run --workspace force-app --config-file code-
analyzer.yml \
  --rule-selector AppExchange --rule-selector Recommended:Security
→ Found 0 violations.
```

Six @SuppressWarnings annotations remain in the source, all in PEAccessQueryService. None of the rules they suppress appear in the AppExchange or Security rulesets. We checked that rather than assuming it: we removed all six annotations, re-ran the gate scan, and it still returned zero violations. We put them back because they keep our broader triage view quiet, so that a genuinely new finding stands out instead of being lost among eight we already know about.

Each is a design decision. Each is explained in the code at the point of suppression as well as here.

PMD.OperationWithLimitsInLoop — five instances

Where: PEAccessQueryService, on applyTabVisibility, applyObjectPermissions, appendAccessWithoutTab, buildObjectGrants, and buildFieldPerms.

Why it fires: each method has a SOQL query inside a for loop.

Why that is deliberate. The loop exists to respect governor limits, not in ignorance of them.

A user's effective permission-set parents include their profile as a permission set, every directly assigned permission set, and every component of every assigned permission set group. In a large enterprise org that is routinely hundreds of Ids — more than a single SOQL bind list should carry. PESoqlChunker.chunk splits the set into batches, and PESoqlChunker.assertQueryBudget checks the resulting query count against the governor limit

before any query runs, throwing a typed LIMIT_EXCEEDED error that the interface renders as a retryable per-section failure.

Hoisting these queries out of the loop is precisely the bug this pattern prevents. A single query with an unbounded bind list either fails at runtime or silently returns partial data, and silently partial data in a permissions tool is worse than an error.

Security impact: none. This is a performance and correctness rule. Every query involved runs in user mode, so object and field permissions are enforced by the platform regardless of how the query is batched.

PMD.CognitiveComplexity and PMD.CyclomaticComplexity — one class-level instance

Where: PEAccessQueryService, class declaration. Cognitive complexity 81 against a threshold of 50; cyclomatic complexity 51 against 50.

Why we accepted it. The complexity here is breadth rather than depth. This class is the query surface for the whole product: six permission columns across four distinct query shapes. During remediation we reduced the per-method figures by extracting mergeCrud, permValues, apiNameByTabName, newAccessWithoutTabRow and seedFieldRows. The class total stays above the threshold because the breadth is inherent to what the class does.

Splitting it to satisfy a number would scatter the absence rule — that a missing ObjectPermissions or FieldPermissions row means no access, never unknown — across several files. That rule has to hold identically in every code path in this product, and keeping it in one place is worth more to us than a metric.

Security impact: none. These are maintainability rules.

PMD.ApexSOQLInjection — resolved, not suppressed

We include this because it appears in our earlier scan history and a reviewer comparing runs may wonder what happened to it.

The original condition. PEUserSearchService.search assembles a SOQL string at runtime to apply optional filters and the keyset cursor predicate.

What we changed. We did not suppress it; we restructured the code until the rule stopped firing:

- The query string is built only from private static final constants declared at the top of the class. Filters are applied by choosing which constant to append. No user-supplied value is ever concatenated into the string.
- All values reach the database through a named bind map passed to Database.queryWithBinds with AccessLevel.USER_MODE.

- The only identifier that varies is the sort column. It comes from an allowlist in PEKeysetCursor.isAllowedSort, defaults to Name, and is then applied by selecting between two constant ORDER BY fragments. The cursor predicate is selected on the same field, so the predicate and the ordering cannot fall out of step with each other.

Three tests cover this: PEUserSearchServiceTest.quoteInTermIsBoundNotInterpolated, PEUserSearchServiceTest.disallowedSortFieldFallsBackToName, and PermissionExplorerTest.disallowedSortFieldIsRejected.

The rule no longer fires and no suppression for it remains in the source.

Two scanner conditions worth explaining

Graph Engine path-evaluation timeouts

Two entry points on PermissionExplorerController exceed the Graph Engine's default 30-second per-path budget:

```
Internal execution error while scanning entry point:
PermissionExplorerController.cls: Path evaluation timed out after
30000 ms
```

These are getObjectDetail and getAppObjects, the two methods that fan out through the full resolver and query stack. This is path explosion in the analyser rather than a finding against the code, but we want to be clear about what it means: those two entry points are reported as **not scanned**, not as passing.

Both are covered by the controls described in the architecture document. PEAccessGate.assertAuthorised() is the first statement in each. Both classes are `with sharing`. All SOQL inside them runs in user mode. Both are exercised by PermissionExplorerControllerTest, PermissionExplorerFlowTest and PEUserDetailServiceTest. We are happy to work with the reviewer if they would like these scanned under a raised timeout.

PMD's JavaScript rules are excluded from our configuration

Our code-analyzer.yml scopes PMD to Apex file extensions:

```yaml
engines:
  pmd:
    file_extensions:
      apex:
        - .cls
        - .trigger
```

PMD's JavaScript ruleset predates ES modules and Lightning Web Components. Run against our LWC source it produced 95 GlobalVariable violations, every one of them a standard browser API — window, document, URLSearchParams, setTimeout — which PMD treats as an undeclared global. There were no true positives among the 95.

We lint the Lightning Web Components separately with ESLint and @salesforce/eslint-config-lwc, through `npm run lint`, which understands browser globals and LWC decorators natively.

The code-analyzer.yml file is part of the submitted source and can be inspected.

Declarations

- **No callouts.** Searching the package source for HttpRequest, `new Http()`, and `callout:` returns zero matches in each case. No Named Credentials, External Credentials, Remote Site Settings, or Auth Providers are present.
- **No credential storage.** No OAuth tokens, session tokens, API keys, or passwords are stored in any form.
- **No data stored or transmitted outside the Salesforce Platform.**
- **No `without sharing`.** All 16 implementation classes are `public with sharing`.
- **No SYSTEM_MODE.** There is no AccessLevel.SYSTEM_MODE and no `WITHOUT USER_MODE` anywhere in the source.
- **No dynamic Apex.** No Type.forName, and no deserialisation of user-controlled input that drives execution.